RUB

RUHR-UNIVERSITÄT BOCHUM

I/O DEVICES

Computer Architecture

Admin

Exam

- Date: 13.02.2026
- Time: 14:30 – 16:30
  - Be there at 14:15!
- Bring
  - Student ID
  - Pens
  - Cheat sheet
- English
- Material:
  - Lectures / Tutorials / Exercises

Agenda

- I/O Modules
- Programmed I/O
- Interrupt-Driven I/O
- Direct Memory Access
- Advanced Topics

I/O Modules

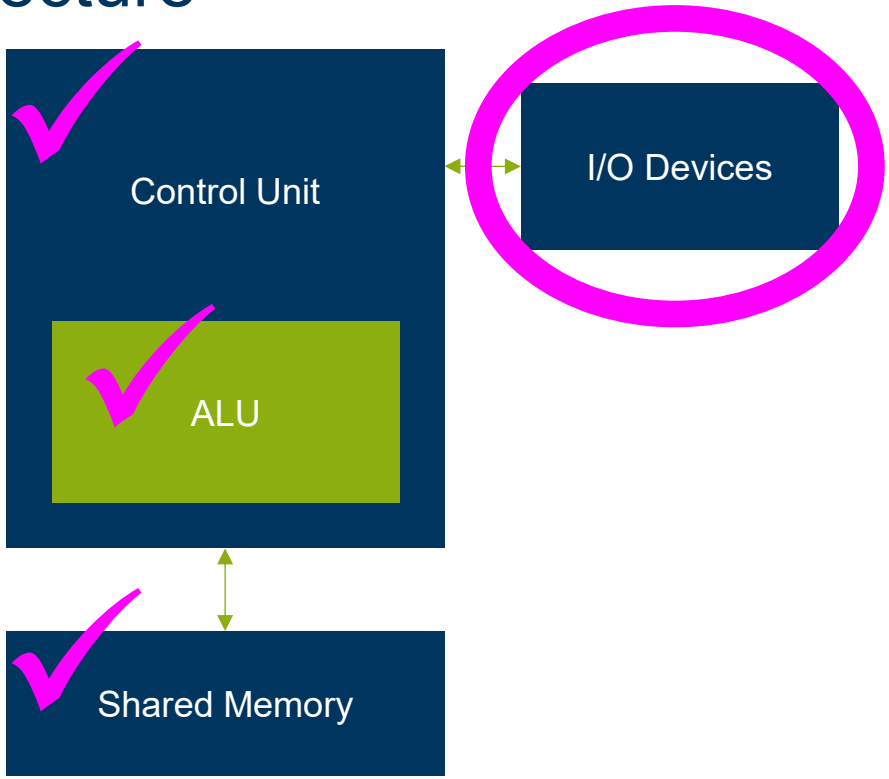
Recap: Von Neumann Architecture

So far we have discussed

- ALU
- Control Unit
- Memory

This lecture

- I/O Devices



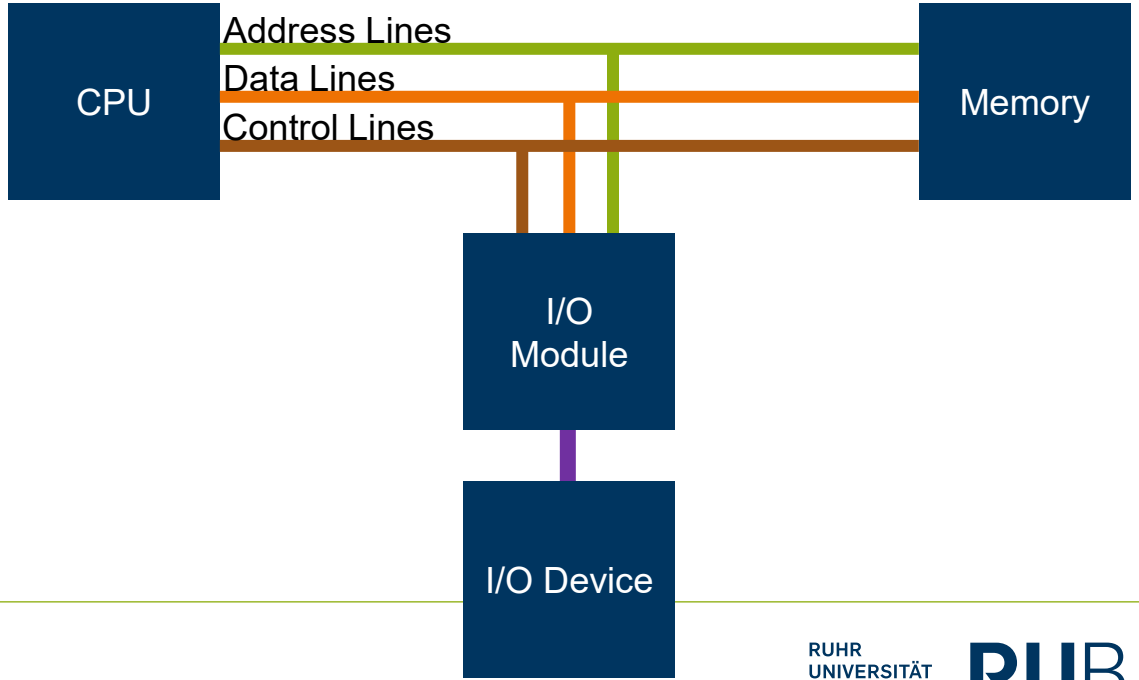
Device Types

- Human readable
  - Keyboard
  - Screen
  - Printer
- Machine readable
  - Disk and tape drives
  - Sensors and actuators
- Communication
  - Network interfaces

I/O Modules

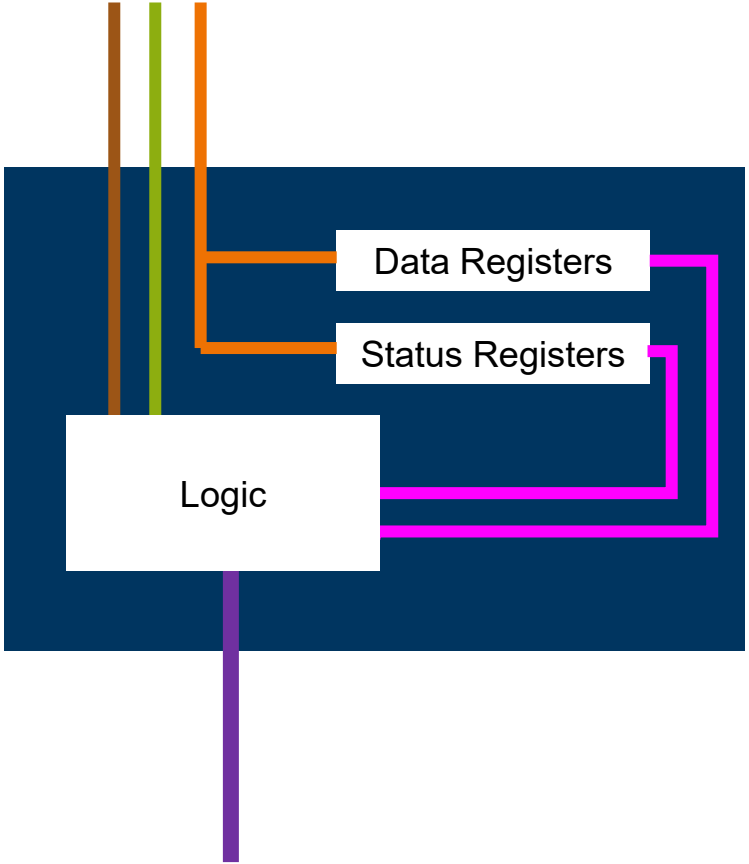
Interface between I/O devices and the system bus

- Large variety of devices
- Communication speeds
- Communication patterns
- Data types
- Electrical connections



Module Interaction

- Module connects through the system bus
- Presents addressable registers
  - Data
  - Status
  - Control
- Monitors bus address lines for register addresses
  - Interprets commands
  - Reports device status
  - Transfers data



Programmed I/O

I/O Timing

Problem

- Slow processing
  - Tape rewind
- Timing unknown
  - Key strokes

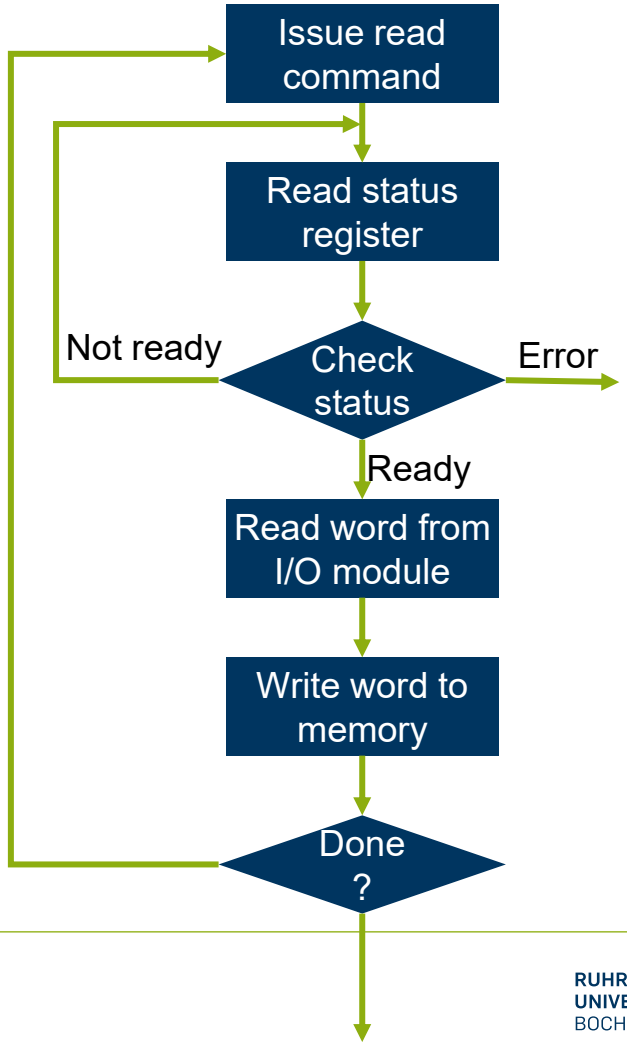
Solution

- Probe device

Programmed I/O

- CPU runs a program that handles the I/O

- Repeatedly check the device status
- Move data when device is ready
- Repeat



I/O Addressing Modes

Memory Mapped I/O

- Single address space shared with memory
- Memory access instructions perform I/O
- Better support for multiple addressing modes
- Limits address space (mostly obsolete)

Isolated I/O

- Separated address space for I/O (port addresses)
- Require separate instructions for I/O – limited addressing
- Easy to distinguish I/O from memory
- More bus lines required

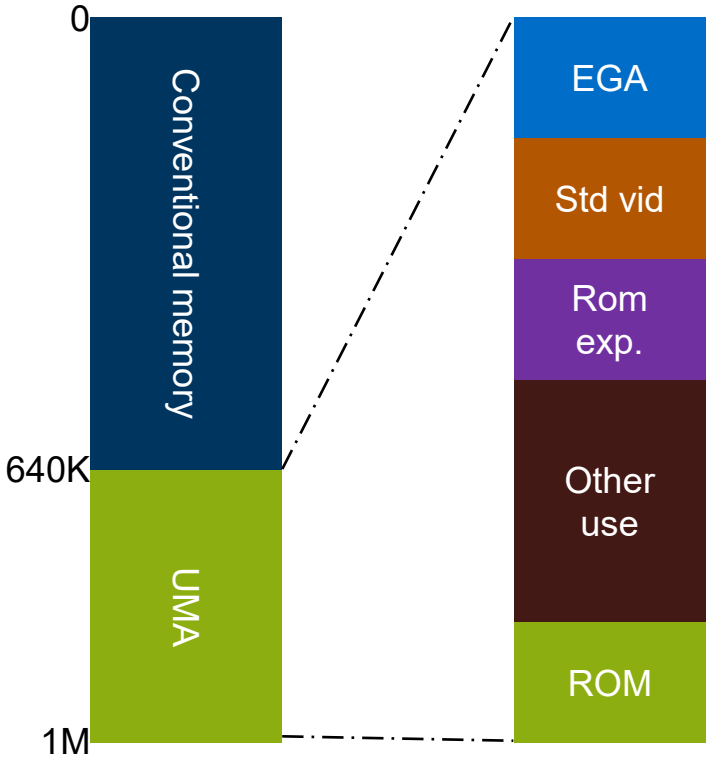
Example – IBM PC

Memory

- 20-bit address space
  - 640KB conventional memory
  - 364KB upper memory area
    - Of which 64KB for video
- Five addressing modes

I/O

- 16-bit port address space
- Two addressing modes
  - Limited registers



Interrupt Driven I/O

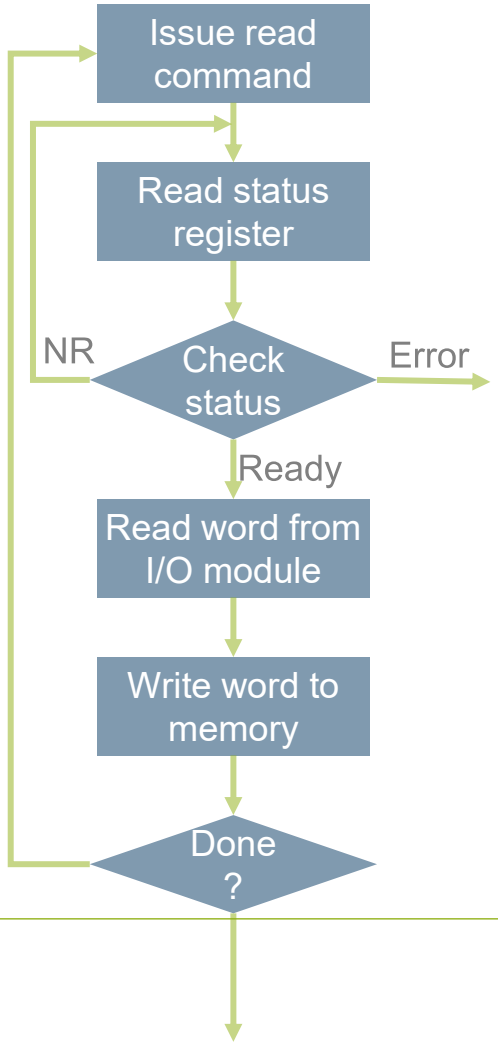
Interrupts

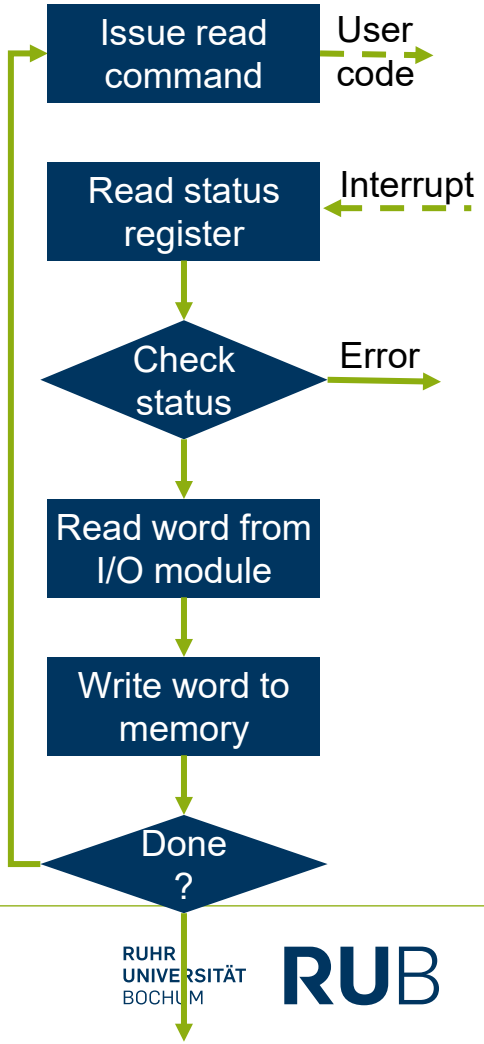
Programmed I/O occupies the CPU

- Wastes CPU resources
- Polling multiple devices is complex
- Hard to interleave with user code

Solution:

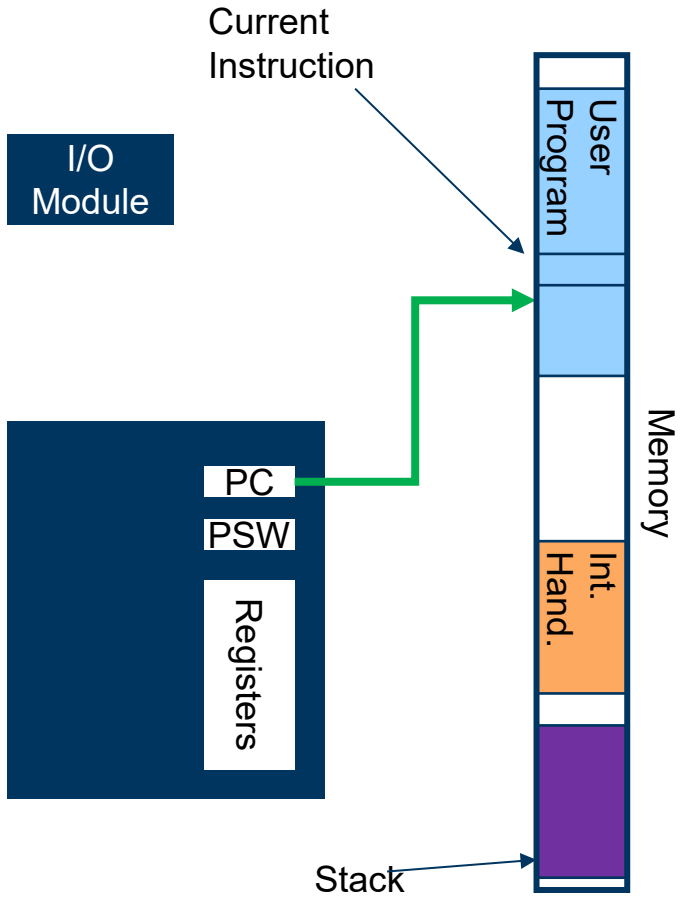
- Device notifies CPU when it's ready
- CPU sends a command to the device
- Continues running user code
- When command is ready the device *interrupts* the normal execution
- CPU handles device





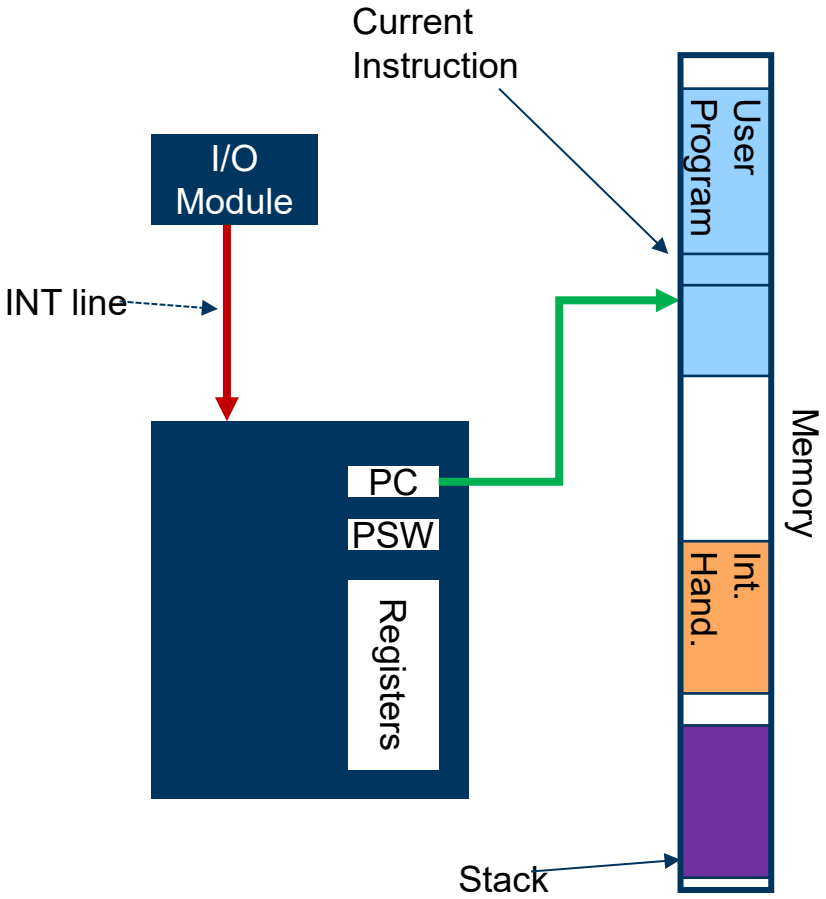
Interrupt Handling

- Device signals interrupt
- Processor finishes execution of current instruction
- Processor acknowledges interrupt
- Processor saves basic hardware state
- Processor changes PC to interrupt handler
- Handler saves remainder of state
- Handler processes interrupt
- Handler restores state
- Handler restores hardware state



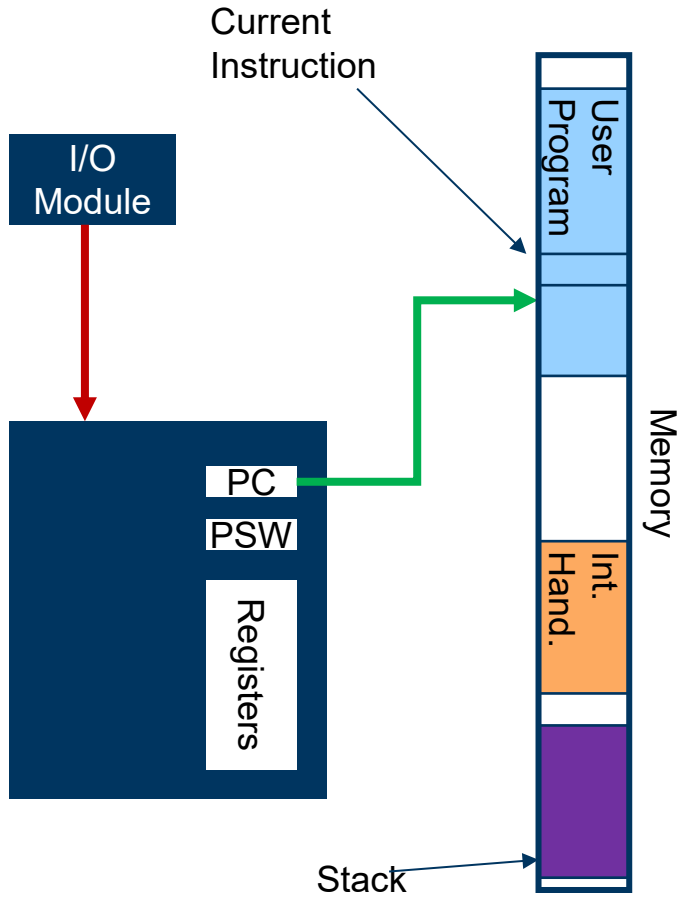
Interrupt Handling

- **Device signals interrupt**
- Processor finishes execution of current instruction
- Processor acknowledges interrupt
- Processor saves basic hardware state
- Processor changes PC to interrupt handler
- Handler saves remainder of state
- Handler processes interrupt
- Handler restores state
- Handler restores hardware state



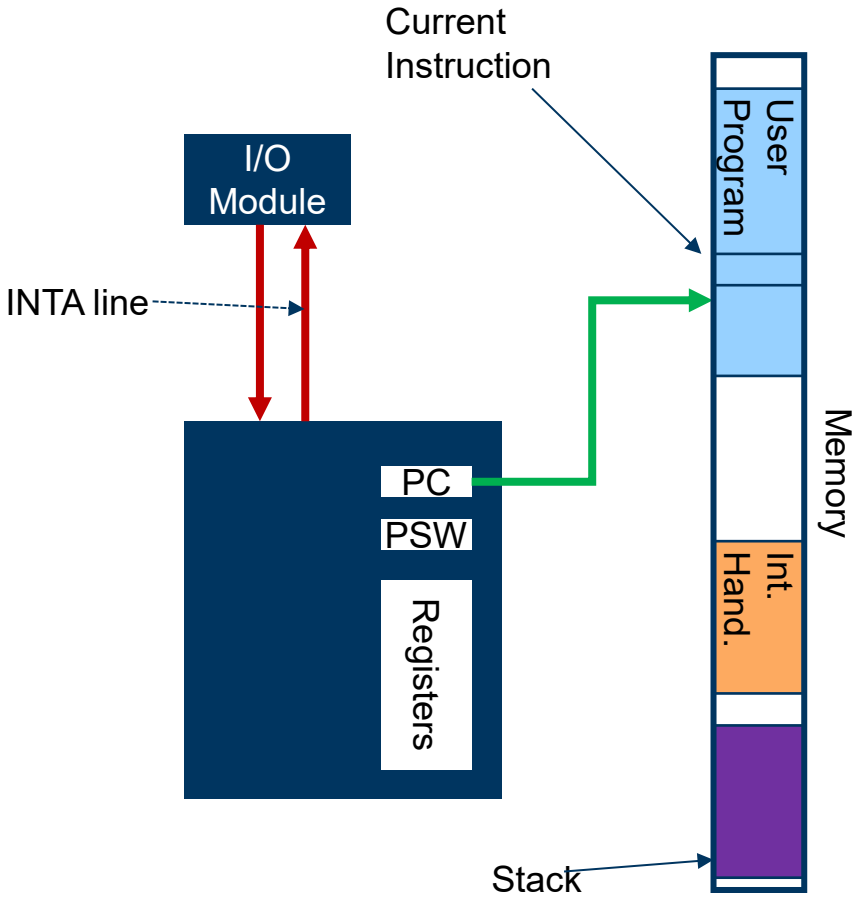
Interrupt Handling

- Device signals interrupt
- **Processor finishes execution of current instruction**
- Processor acknowledges interrupt
- Processor saves basic hardware state
- Processor changes PC to interrupt handler
- Handler saves remainder of state
- Handler processes interrupt
- Handler restores state
- Handler restores hardware state



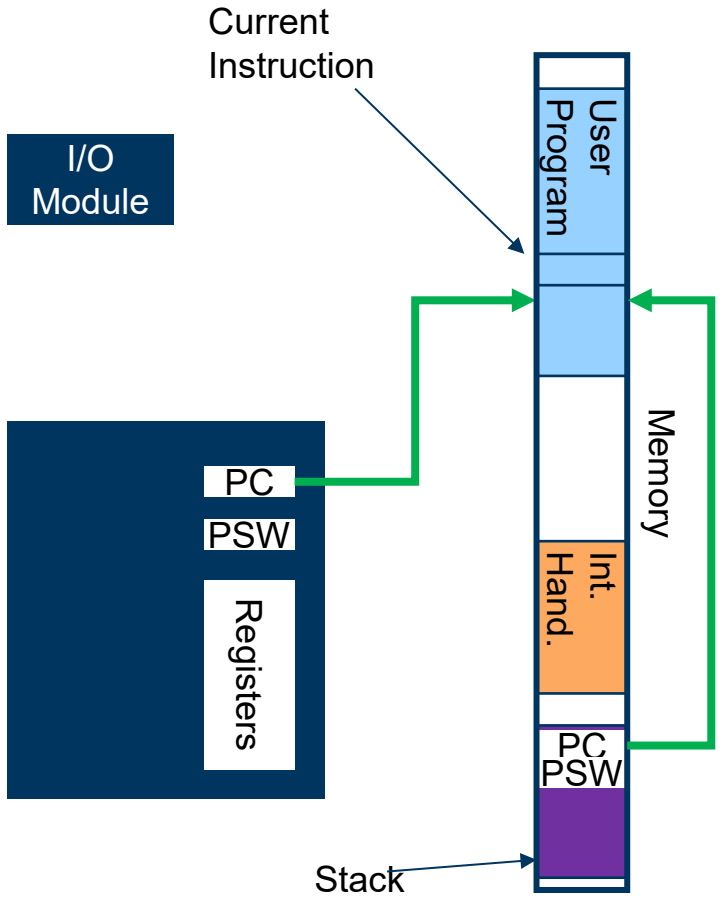
Interrupt Handling

- Device signals interrupt
- Processor finishes execution of current instruction
- **Processor acknowledges interrupt**
- Processor saves basic hardware state
- Processor changes PC to interrupt handler
- Handler saves remainder of state
- Handler processes interrupt
- Handler restores state
- Handler restores hardware state



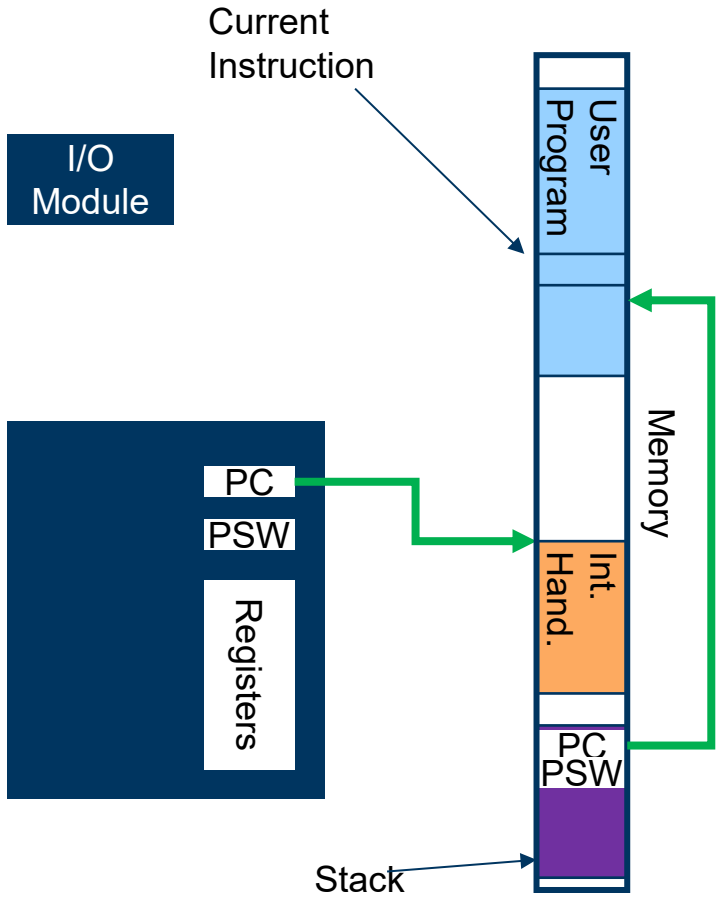
Interrupt Handling

- Device signals interrupt
- Processor finishes execution of current instruction
- Processor acknowledges interrupt
- **Processor saves basic hardware state**
- Processor changes PC to interrupt handler
- Handler saves remainder of state
- Handler processes interrupt
- Handler restores state
- Handler restores hardware state



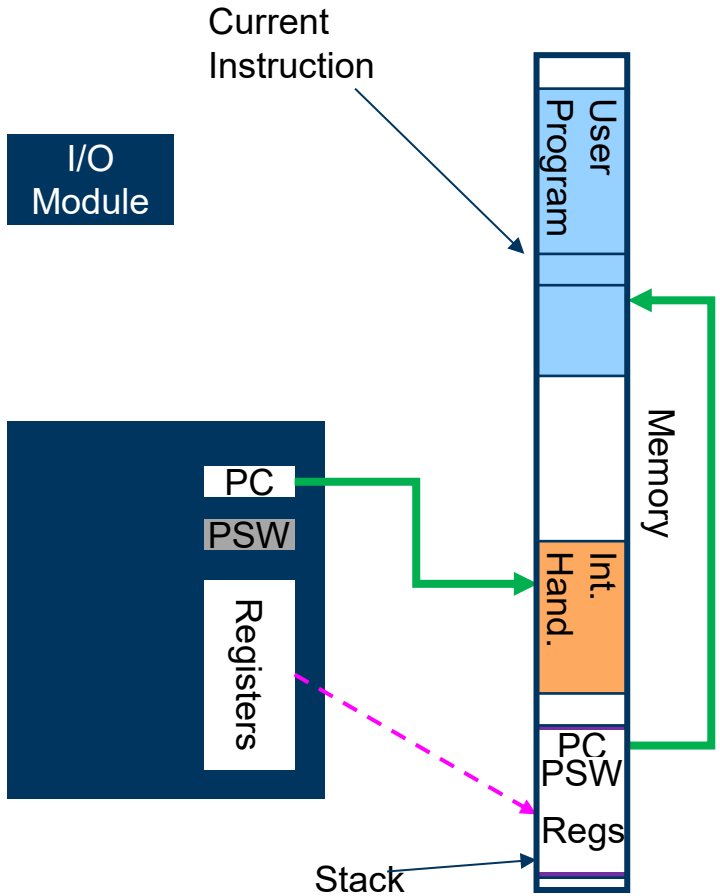
Interrupt Handling

- Device signals interrupt
- Processor finishes execution of current instruction
- Processor acknowledges interrupt
- Processor saves basic hardware state
- **Processor changes PC to interrupt handler**
- Handler saves remainder of state
- Handler processes interrupt
- Handler restores state
- Handler restores hardware state



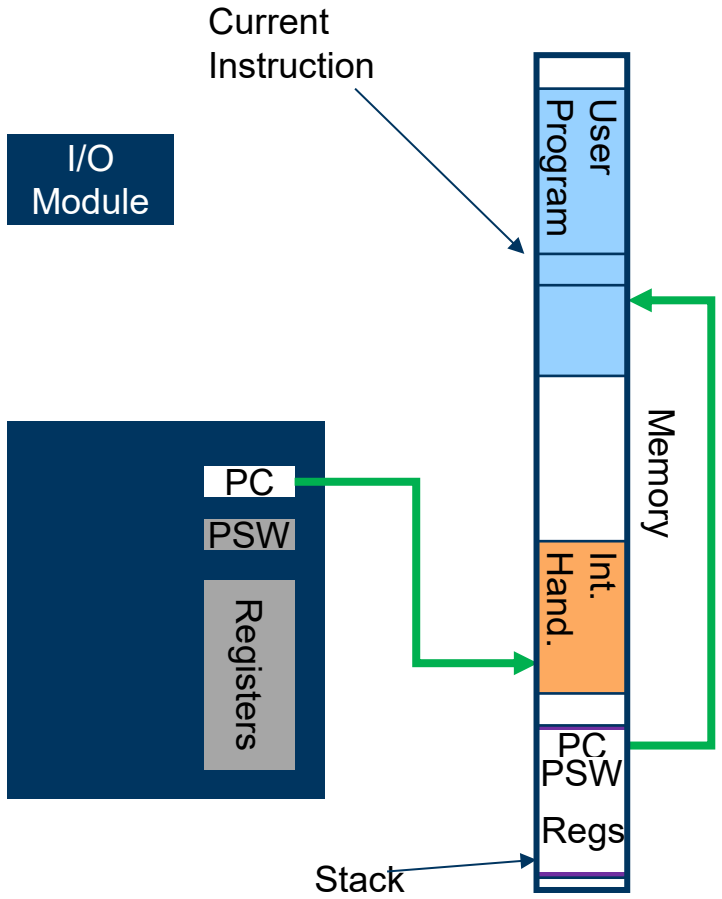
Interrupt Handling

- Device signals interrupt
- Processor finishes execution of current instruction
- Processor acknowledges interrupt
- Processor saves basic hardware state
- Processor changes PC to interrupt handler
- **Handler saves remainder of state**
- Handler processes interrupt
- Handler restores state
- Handler restores hardware state



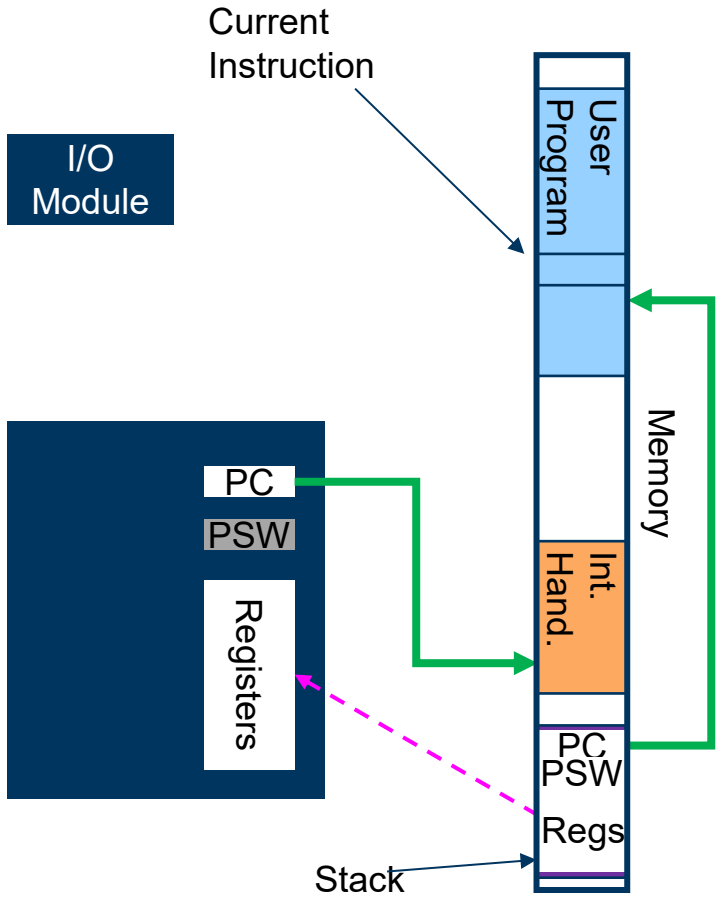
Interrupt Handling

- Device signals interrupt
- Processor finishes execution of current instruction
- Processor acknowledges interrupt
- Processor saves basic hardware state
- Processor changes PC to interrupt handler
- Handler saves remainder of state
- **Handler processes interrupt**
- Handler restores state
- Handler restores hardware state



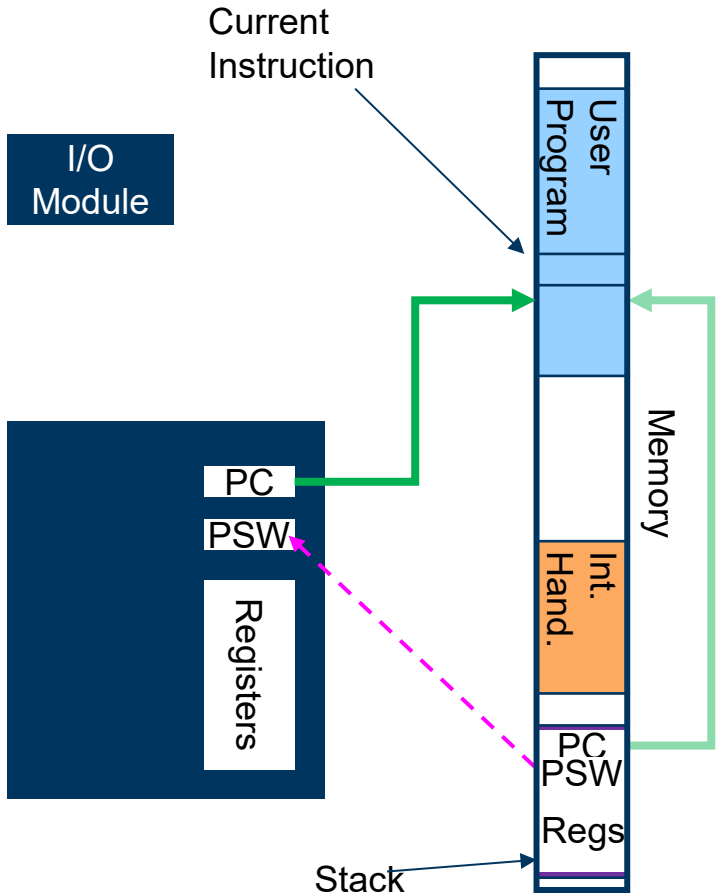
Interrupt Handling

- Device signals interrupt
- Processor finishes execution of current instruction
- Processor acknowledges interrupt
- Processor saves basic hardware state
- Processor changes PC to interrupt handler
- Handler saves remainder of state
- Handler processes interrupt
- **Handler restores state**
- Handler restores hardware state



Interrupt Handling

- Device signals interrupt
- Processor finishes execution of current instruction
- Processor acknowledges interrupt
- Processor saves basic hardware state
- Processor changes PC to interrupt handler
- Handler saves remainder of state
- Handler processes interrupt
- Handler restores state
- **Handler restores hardware state**



Identifying Interrupt Source

When there are multiple devices, how does the CPU know which generated an interrupt?

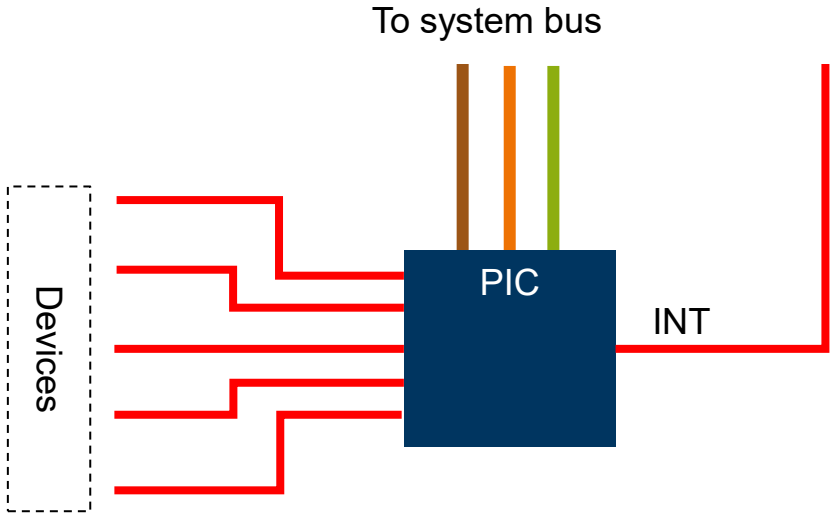
Approach 1: Multiple INT lines

Approach 2: Poll all devices

Approach 3: Interrupt controller

Interrupt controller

- An I/O module that helps the CPU handle interrupts
- Multiple input interrupt lines
- When a device signals an interrupt
  - PIC signals INT line
  - Processor queries PIC for device ID
  - Processor switches to device-specific handler
- Prioritizes interrupts



- INTEL 82C59A can handle 8 interrupt sources
- Can cascade to eight 82C59As

Direct Memory Access

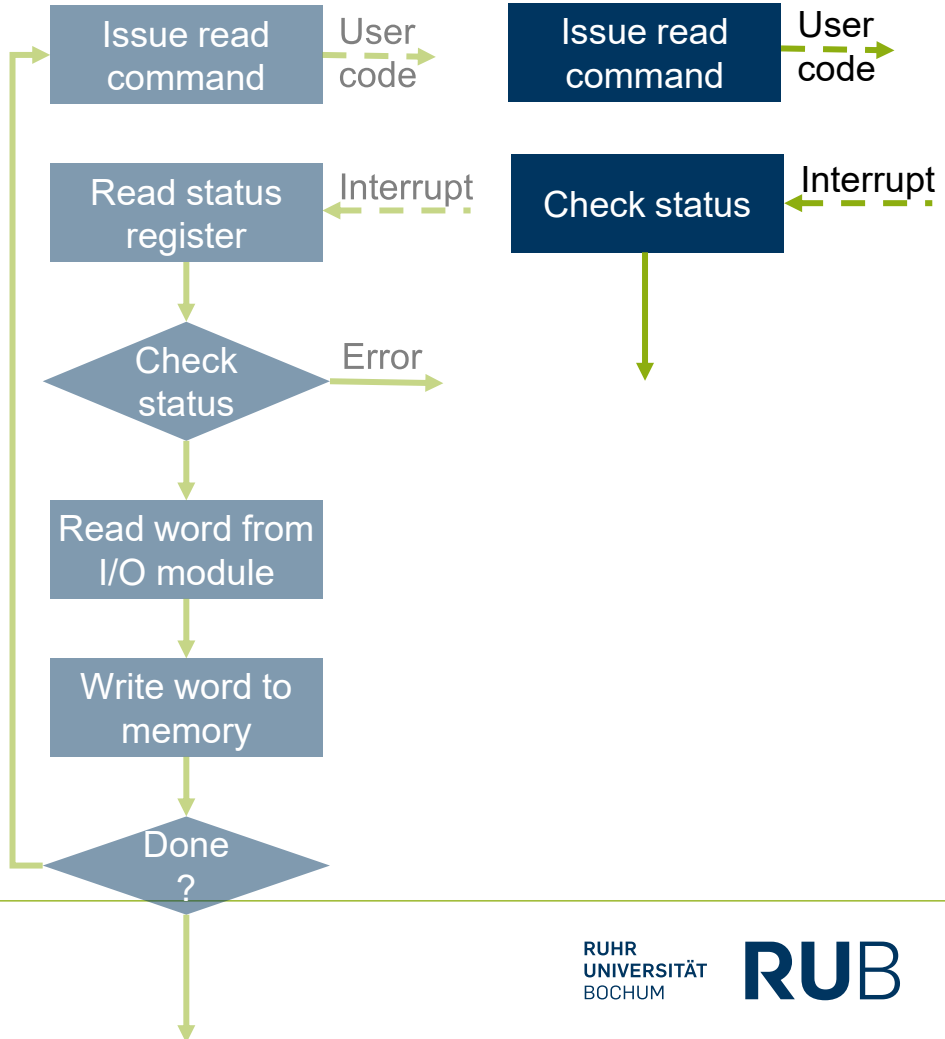
Can we do better?

Interrupts do not solve everything

- Overhead due to interrupt set-up and handling
- Data goes twice over the bus

Solution:

- Let the I/O module copy data to memory

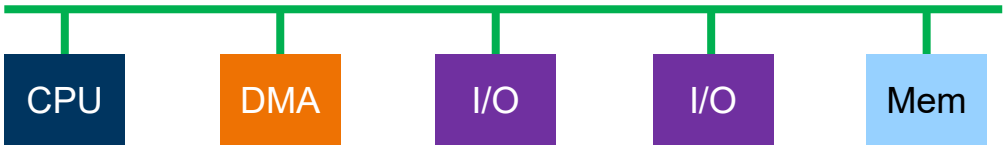


Direct Memory Access (DMA)

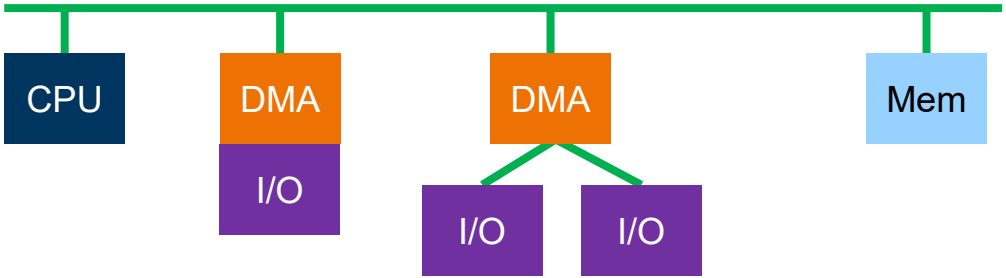
DMA controller transfers data between devices and memory

- Needs independent access to the system bus
- Cycle steeling
  - DMA controller raises a HRQ signal on the bus
  - Processor stops executing and raises HLDA
  - DMA controller uses the bus
  - DMA controller clears HRQ
  - Processor resumes executing
- More modern devices – processor continues executing as long as it does not need the bus

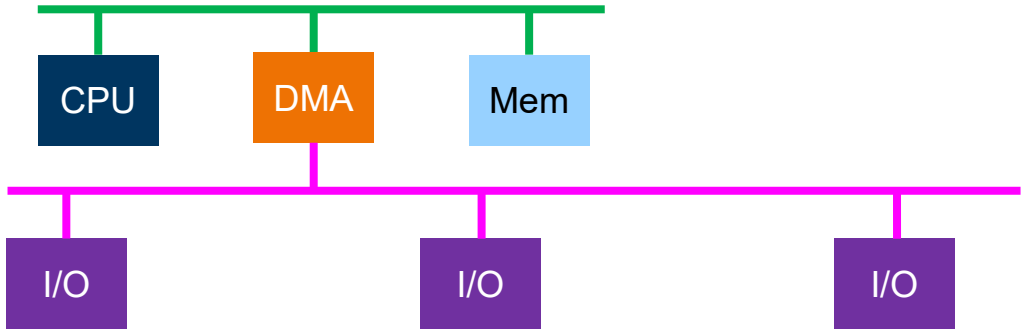
DMA Configurations - Detached



DMA Configurations - Integrated



DMA Configurations – I/O Bus



Advanced Topics

DMA and caches

DMA does not work well with caches

- Need to invalidate cache on DMA write to memory
- Need to write-back on DMA read from memory

Solution:

- Place the DMA controller above the last-level cache
- DMA participates in cache coherency

New problem:

- DMA access patterns kill cache performance

Direct Cache Access

Cache is final destination of I/O

Intel implementation: DDIO (Direct Data I/O)

- Aimed at network decies

Device read:

- Cache miss: do not fill cache
- Cache hit: evict from cache

Device write:

- On cache hit – update cache line
- On cache miss - allocate a cache line without reading from memory

I/O processors

An extension of DMA

- The I/O module is itself a processor
  - Main CPU sends a program to I/O module
  - Interrupts only when program finishes/fails

- Nomenclature:
  - I/O channel – without local memory
  - I/O processor – with local memory

I/O Processors

Summary

Development of I/O handling

- I/O modules abstract over I/O devices
- Interrupt driven I/O shifts synchronization to I/O module
- DMA shifts memory access to I/O module
- I/O processors shift some computation